UNIVERSITÉ CÔTE D'AZUR

FPGA

TP 1 : Programmation de FPGA + synthèse

Réalisé par :

BOURAOUI Amira

BARBE Maxime

CHIBANE Sara

Encadré par :

Mme Belleudy

1. INTRODUCTION :

Ce TP a pour but de nous initier à la programmation des FPGA, en particulier ceux de la famille Cyclone III. Nous allons apprendre à concevoir, simuler et implémenter des circuits numériques sur ce matériel. Le TP est organisé en deux grandes parties et couvre les points suivants :

- Architecture du Cyclone III : étude de la structure du FPGA et de ses composants (cellules logiques, mémoires, etc.).
- Synthèse et temps de réponse : implémentation d'opérateurs arithmétiques (additionneur, multiplicateur) et analyse de leurs performances selon la configuration.
- Filtrage vidéo : réalisation d'un filtre vidéo simple et implémentation sur le FPGA.

1.1 Exercice 1 : Architecture et caractéristiques techniques

La famille de FPGA utilisé est Cyclone V. Etudions l'architecture du composant et des cellules élémentaires :

Architecture d'un FPGA Cyclone V :

L'architecture d'un FPGA Cyclone V repose sur un réseau organisé de Logic Array Blocks (LABs) contenant des éléments logiques (LEs). Autour de ces blocs se trouvent diverses ressources : blocs mémoire, blocs DSP, PLL et entrées/sorties configurables.

1. Logic Array Blocks (LABs)

Un Cyclone V contient de nombreux LABs. Chaque LAB regroupe plusieurs éléments logiques (LEs) (généralement 10 par LAB). Les LABs sont disposés en matrice et reliés entre eux par un réseau d'interconnexion hiérarchique, permettant la création de circuits logiques complexes.

2. Éléments logiques (Logic Elements – LEs)

Les LEs sont les unités logiques de base.

Chaque LE comprend :

- une LUT (Look-Up Table) à 4 entrées,
- un registre (flip-flop),
- des ressources dédiées aux opérations logiques et arithmétiques simples.

3. Réseau d'interconnexion programmable

Les LABs et les autres blocs (mémoire, DSP, I/O) sont connectés via un réseau de routage programmable. Ce réseau comprend des lignes locales et globales permettant d'acheminer les signaux à travers tout le FPGA.

4. Blocs mémoire embarquée

Les Cyclone V utilisent principalement des blocs M10K (10 Kbits chacun). Ces blocs peuvent servir de RAM simple ou double port, FIFO ou ROM. Ils sont répartis dans la puce et accessibles via le réseau d'interconnexion.

5. Blocs DSP (Digital Signal Processing)

Les FPGA Cyclone V intègrent des blocs DSP optimisés pour :

- la multiplication,
- l'addition,
- l'accumulation et autres traitements numériques.

Ils sont utilisés dans le traitement vidéo, audio, communications ou signal.

6. Blocs PLL (Phase-Locked Loops)

Plusieurs PLL intégrées permettent de générer et ajuster des signaux d'horloge de fréquences différentes, tout en assurant la synchronisation interne.

7. Blocs d'Entrées/Sorties (I/O)

Les ports d'E/S du Cyclone V sont hautement configurables et compatibles avec de nombreux standards comme :

- LVTTL,
- LVCMOS,
- SSTL,
- HSTL, etc.

8. Configuration SRAM et reprogrammabilité

Le Cyclone V est basé sur une architecture SRAM, ce qui le rend totalement reprogrammable.

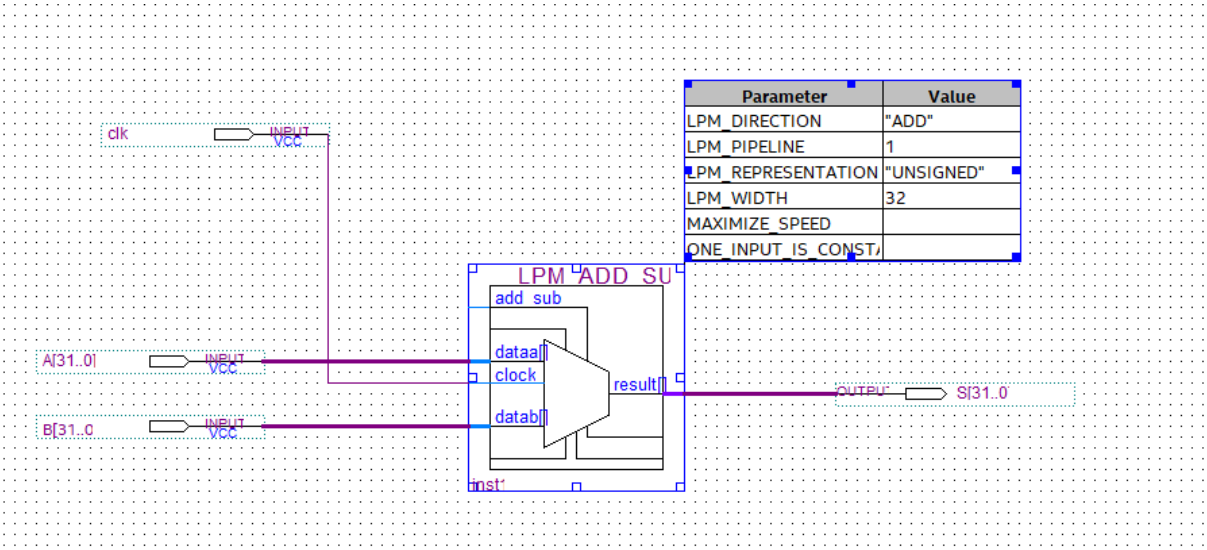
La configuration (LUT, routage, mémoire…) est stockée dans des cellules SRAM, permettant les mises à jour ou changements après fabrication.

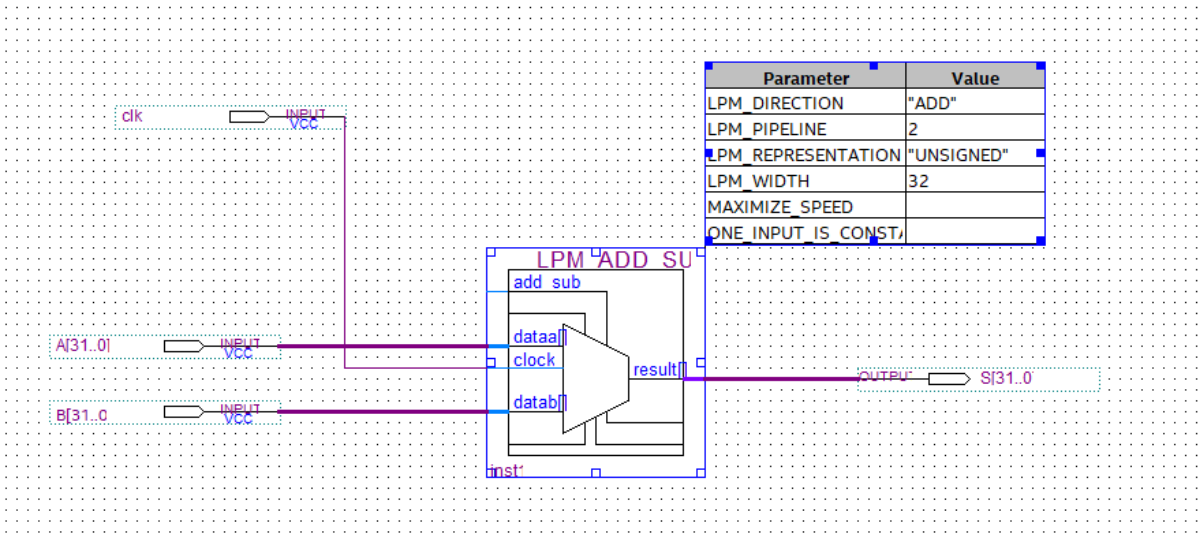
2. Exercice 2 : Synthèse FPGA et temps de réponse

Notre travail consiste à évaluer l'impact du pipeline sur le temps de réponse et la consommation de ressources d'un additionneur 32 bits, à comparer les performances obtenues avec différentes configurations de pipeline (sans pipeline, avec 2 étages et avec 4 étages), à analyser les particularités de l'implémentation de l'addition en virgule flottante sur FPGA, à observer l'effet de ces configurations sur le mapping et le temps de

réponse, puis à synthétiser un filtre vidéo simple basé sur une convolution et à en évaluer les performances en termes de temps de traitement.

2.1. Dans cette partie, l'objectif est d'analyser comment le temps de réponse et la complexité d'un additionneur 32 bits non signé évoluent en fonction du niveau de pipeline. Il s'agit également de mesurer l'impact du pipeline sur les performances et l'utilisation des ressources, en particulier le temps de réponse et le nombre d'éléments logiques (LE). Enfin, un tableau comparatif présentera les temps de réponse entre une version pipeline 1 et des versions pipelinee à 2 et 4 étages.





Nous allons résumer les résultats obtenues dans un tableau :

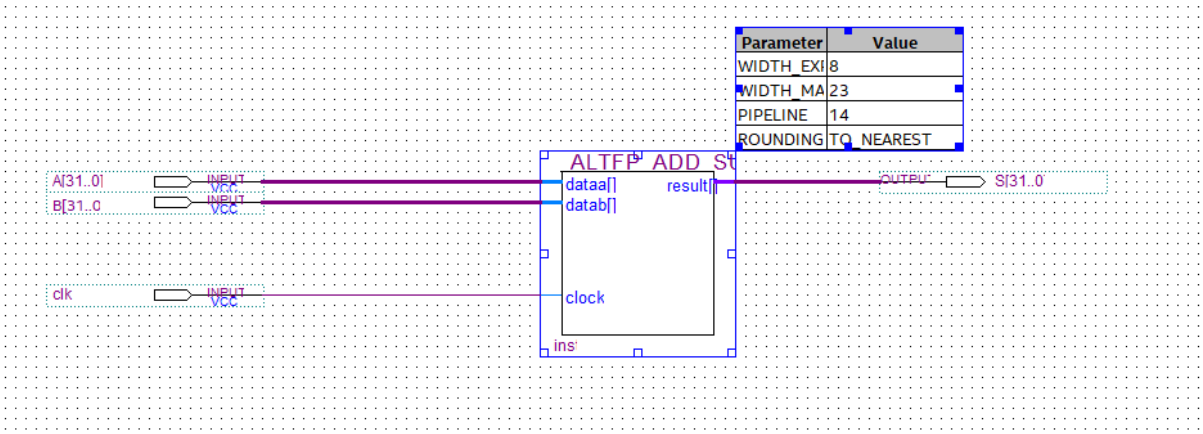
	Pipeline 1	Pipeline 2	Pipeline 4
LE	16	33	67
CLK	12 ,6	13,4	13,4
HOLD	1	0,7	0 ,7
MIN	8,5	8,3	8,3

SETUP	5,4	7,3	7,3
Somme	27,5	42,6	42,6

Conclusion :

L'analyse montre que l'augmentation du nombre d'étages de pipeline accroît nettement l'utilisation des ressources logiques (de 16 à 67 LE entre 1 et 4 étages). En revanche, le temps d'horloge ne s'améliore pas : il passe de 12,6 ns à 13,4 ns, indiquant une absence de gain de performance. Les contraintes temporelles (SETUP, HOLD, MIN) varient légèrement, sans effet positif notable. Le pipeline 4 n'apporte aucun avantage par rapport au pipeline 2. Ainsi, le pipelining augmente la complexité sans réduire le temps de réponse. La version à 1 étage offre donc le meilleur compromis entre performance et ressources.

2.2. Nous abordons maintenant l'étude d'un additionneur en virgule flottante. À la différence de l'addition entière, cette opération est plus complexe en raison de la représentation des flottants, composée d'un signe, d'une mantisse et d'un exposant. L'addition se déroule en plusieurs étapes : d'abord l'alignement des mantisses, où l'on identifie l'exposant le plus grand et on décale la mantisse du nombre ayant l'exposant le plus petit jusqu'à égalisation ; ensuite l'addition des mantisses alignées comme des entiers ; puis la normalisation du résultat en cas de dépassement ou de bits non significatifs en tête, ce qui implique un ajustement de l'exposant ; enfin un éventuel arrondi pour respecter la précision limitée du format flottant. Nous pouvons maintenant passer à l'analyse des configurations de mapping et du temps de réponse associé.



Nous allons résumer les résultats obtenues dans un tableau :

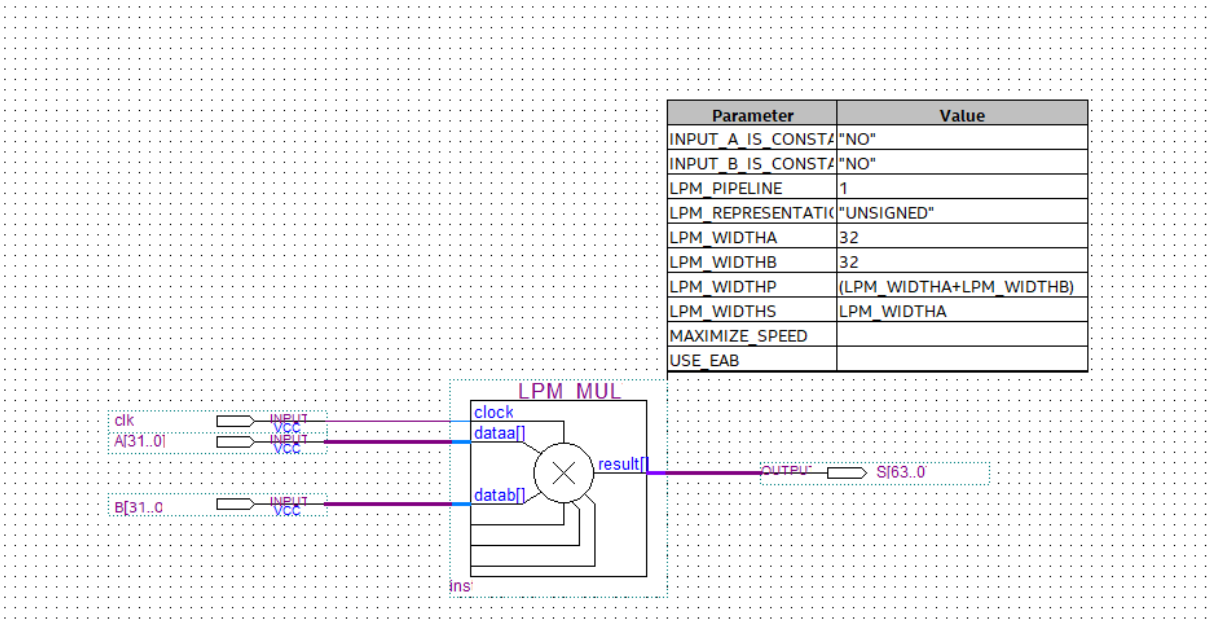
	Pipeline 14
LE	465
CLK	16,9
HOLD	1,6
MIN	8,4

SETUP	8,6
Somme	35,5

Conclusion :

L'additionneur en virgule flottante présente une complexité nettement supérieure à celle de l'addition entière, comme en témoigne l'utilisation élevée des ressources (465 LE). Le temps d'horloge atteint 16,9 ns, reflétant la succession d'étapes nécessaires à l'alignement, l'addition et la normalisation des mantisses. Le délai total de 35,5 ns montre que la profondeur du pipeline ne compense pas totalement la complexité de l'opération. Cette architecture reste donc plus coûteuse en ressources et en temps de réponse, mais indispensable pour le traitement précis des nombres réels.

2.3. Nous proposons d'étudier l'opérateur de multiplication en évaluant, pour différentes implémentations utilisant soit un bloc multiplieur dédié soit des LUT, le temps de réponse ainsi que le taux d'occupation du FPGA.



Nous allons résumer les résultats obtenues dans un tableau :

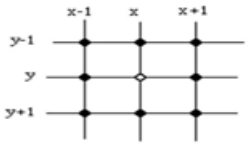
	Pipeline 14
LE	57
CLK	19
HOLD	0,97
MIN	9,3
SETUP	8,6
Somme	37,87

Conclusion :

L'étude de la multiplication montre une utilisation modérée des ressources (57 LE) grâce au recours à un bloc multiplieur dédié. Le temps d'horloge atteint 19 ns, indiquant une opération plus coûteuse que l'addition entière mais mieux maîtrisée que l'addition flottante. Le délai total de 37,87 ns reste cohérent avec la complexité de l'opération. Cette implémentation offre ainsi un bon compromis entre performance et occupation du FPGA.

2.4. On souhaite réaliser un filtre vidéo. Pour cette application, pour chaque pixel d'une image, on applique le traitement suivant : $g(x,y)=\Sigma i \Sigma j\ h(i,j)*I(x+i, y+j)$ avec i et j variant de -n à +n.

Exemple de filtre :



$$h=1/10*\begin{vmatrix}1 & 1 & 1\\ 1 & 2 & 1\\ 1 & 1 & 1\end{vmatrix}$$

On va modéliser ce filtre.

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL; -- Remplace std_logic_unsigned (non standard)

ENTITY filtre_video IS
    PORT (

        p00, p01, p02, p10, p11, p12, p20, p21, p22 : IN INTEGER range 0 to 255;
        result    : OUT INTEGER range 0 to 255
    );

END filtre_video;

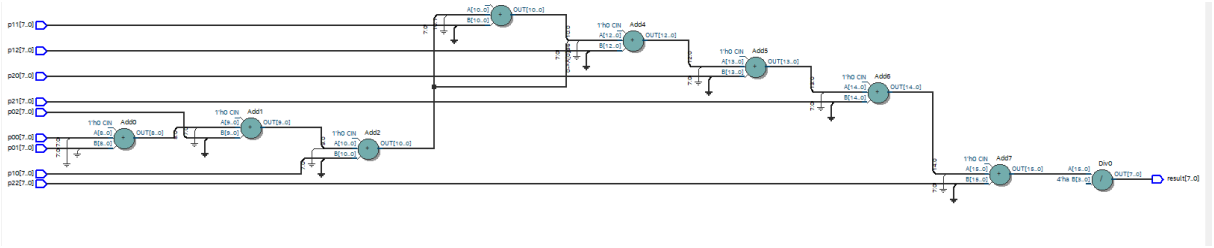

ARCHITECTURE arch OF filtre_video IS

BEGIN

    result <=  (p00 + p01 + p02 + p10 + (2 * p11) + p12 + p20 + p21 + p22)/10;


END arch;
```

Ainsi on obtient ceci :



CONCLUSION :

Ce TP nous a permis de nous familiariser avec la programmation sur FPGA Cyclone V, de concevoir et d'optimiser l'architecture des cellules élémentaires, et d'analyser l'impact des opérateurs arithmétiques, tels que l'addition et la multiplication, sur le nombre de cellules logiques (LE) utilisées et sur le temps de réponse (Tpd). Nous avons étudié différentes configurations, avec ou sans pipeline, et observé les gains de performances grâce à des outils comme le Classic Timing Analyzer de Quartus.

Nous avons étudié la synthèse matérielle et le mapping à travers différents opérateurs sur entiers et flottants. La dernière partie du TP a consisté à implémenter un filtre vidéo sur FPGA afin d'observer l'impact du parallélisme sur la vitesse d'exécution. Ce travail montre clairement que les choix d'architecture influencent fortement les performances, l'utilisation des ressources et les temps de traitement.